

INFORME DE TRABAJO PRÁCTICO INTEGRADOR – ETAPA II

Asignatura: Programación II – POO

Profesor: David Roco

Equipo: Equipo 2

Capitán: Leonel Fachinelli

Integrantes: Maximiliano Reinoso, Xavier Pappalardo y Lautaro Gómez

Roles de esta entrega:

Redaccion, entrega de informe y coordinación general: Leonel Fachinelli

Hardcodeo y backend: Maximiliano Reinoso y Xavier Pappalardo

Diseño de UML y Testeo: Lautaro Gómez

1. ANÁLISIS DEL PROBLEMA Y JUEGO ELEGIDO

En esta segunda parte del proyecto, consistió en reutilizar y extender la estructura ya hecho base de la baraja española. Todo esto con el fin de jugar a través de la consola.

Seleccionamos el juego **Casita Robada**. Las razones de esta elección es que se necesita un buen manejo de datos, además que es un juego dentro de todo simple y divertido de jugar. Esto representa un escenario óptimo para aplicar herencia, encapsulamiento, polimorfismo y separación de responsabilidades en POO.

2. REGLAS ESPECÍFICAS DEL JUEGO IMPLEMENTADO

El sistema respeta y ejecuta de forma automatizada las reglas clásicas de la Casita Robada:

- **Objetivo:** Acumular la mayor cantidad de cartas posible en la casita personal.
- **Preparación e Inicio:** Se valida un mínimo de 2 jugadores en la mesa. Se mezcla el mazo, se colocan 4 cartas iniciales boca arriba en la mesa y se reparten 3 cartas ocultas a la mano de cada jugador.
- **Dinámica de Turnos:** En cada turno el jugador elige una carta de su mano. El sistema evalúa numéricamente las coincidencias existentes:
 1. *Robo de las cartas centrales:* Si el número de la carta coincide con alguna del suelo, el jugador se lleva ambas a su casita.

2. *Robo de Casita Rival*: Si el número coincide con la carta del tope de la casita de un oponente, el jugador le roba la casita completa y la suma a la suya.
3. *Conflicto de Opciones*: Si la carta coincide simultáneamente con el suelo y con la casita rival, se despliega un menú interactivo para que el usuario elija estratégicamente qué acción realizar.
4. *Descarte*: De no haber ninguna coincidencia, la carta se deposita boca arriba en el suelo.

Mecánica de Recarga: Cuando todos los jugadores se quedan sin cartas en la mano, el sistema valida si quedan elementos en el mazo original y reparte automáticamente una nueva mano de 3 cartas a cada uno.

Fin de Partida y Condición de Victoria: Al finalizar los turnos estipulados o vaciarse las colecciones, el juego realiza el conteo final evaluando el tamaño (`size()`) de las casitas. Gana el jugador con mayor volumen de naipes acumulados, declarándose un empate en caso de paridad absoluta.

3. ARQUITECTURA DE SOFTWARE Y CLASES UTILIZADAS

Para cumplir con los criterios de modularidad exigidos, divididos el proyecto en dos packages ([Parte1](#) y [Parte2](#)):

Clases Reutilizadas ([Parte01](#))

- **Cartas**: Crea el objeto carta, con sus respectivos atributos: número (`int`) y palo (`String`).
- **Baraja**: Es el encargado de gestionar las cartas. Implementa la lógica para mezclar el mazo (`barajar()`), controla la extracción secuencial de los naipes (`sacarSiguiente()`) y resuelve la distribución equitativa de las manos al inicio de la partida (`repartirMano()`).
- **Jugador**: Representa a cada participante de la partida. Su función es almacenar la identidad del usuario (nombre) y custodiar el conjunto de cartas privado (`mazo`) que le ha sido asignado para competir.
- **Mesa**: Actúa como el escenario central y controlador de la partida. Se encarga de coordinar el entorno de juego y mantener el registro activo de los participantes que interactúan en la sesión ([lista de jugadores](#)).

Clases Nuevas e Incorporaciones ([Parte02](#))

- **Juego (Clase Abstracta)**: Define la plantilla general y el estado mínimo que requiere cualquier juego de cartas. Establece los atributos base (`ganador`, `baraja`, `mesa`, `nombre` y `reglamento`) para permitir que el software se pueda extender fácilmente en el futuro con otros modos de juego.
- **CasitaRobada (Clase Concreta)**: Hereda de `Juego` y actúa como el motor principal de la partida. Controla el bucle principal de la simulación (`jugar()`), gestiona las interacciones del usuario por consola (`turno()`), evalúa las capturas posibles (`menu()`) y ejecuta la lógica específica para capturar naipes (`ejecutarRoboMesa()` y `ejecutarRoboCasita()`).

- **ReglasJuego (Clase Abstracta) y Reglas_Casita_Robada (Clase Concreta):** Componentes diseñados bajo polimorfismo para aislar el reglamento del flujo principal del programa. Su función es separar la visualización y el control de las normas para que el código sea más modular.
- **Main:** Es el punto de entrada de la aplicación. Se encarga de inicializar los componentes del sistema, inyectar las dependencias necesarias mediante métodos de acceso (**setters**) e iniciar la ejecución de la partida.

4. JUSTIFICACIÓN DEL USO DE **ArrayList**

El uso de colecciones dinámicas de la biblioteca estándar de Java (**java.util.ArrayList**) fue crítico para el éxito del proyecto, utilizándose en:

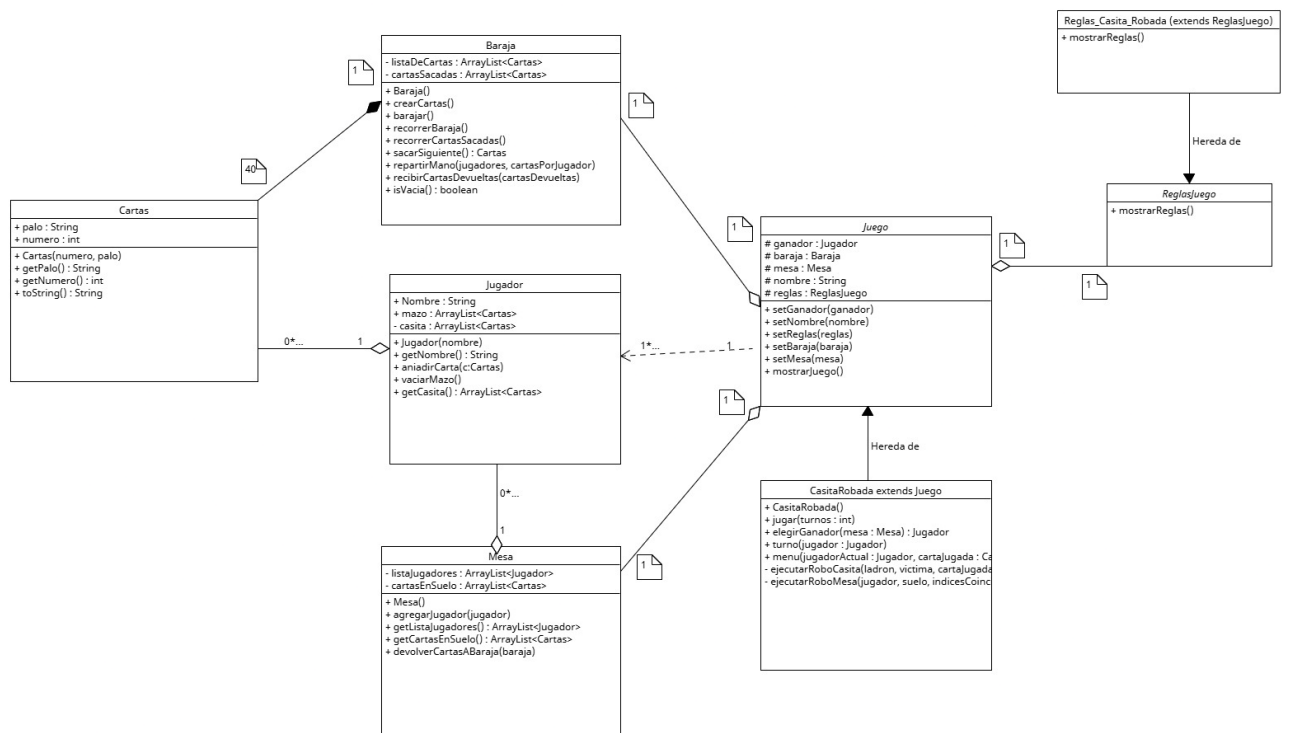
1. **mesa.getCartasEnSuelo():** Administra el flujo indeterminado de descarte y recolección de cartas en el tablero. Permite manejar un volumen variable de elementos y actualizar de forma segura las posiciones de los naipes que quedan disponibles para ser capturados.
2. **jugador.getCasita():** Modela la pila acumulativa de cartas robadas. Su tamaño crece dinámicamente al agregar elementos individuales (**add()**) o transferir lotes completos de naipes (**addAll()**) cuando se realiza un robo a la casita de un rival.
3. **jugador.mazo:** Representa la mano actual del jugador. Permite reducir el tamaño de la colección de manera limpia y segura mediante la eliminación de la carta seleccionada (**remove(index)**) cada vez que el usuario realiza una jugada.

5. ANÁLISIS DE PATRONES DE DISEÑO APLICABLES

Evaluando la estructura final, identificamos y justificamos los siguientes patrones:

- **Template Method (Patrón de Comportamiento):** Implementado entre **Juego** (clase abstracta) y **CasitaRobada** (clase concreta). La clase base define el esqueleto estructural y los atributos comunes del juego, delegando el control del flujo y algoritmo específico del turno (**jugar()**) a la subclase.
- **Strategy (Patrón de Comportamiento):** Aplicado en el desacoplamiento de las normas mediante **ReglasJuego**. Permite cambiar dinámicamente las reglas del juego en tiempo de ejecución inyectando una estrategia diferente (**setReglas()**), lo que facilita añadir nuevos modos de juego (como Truco o Siete y Medio) sin alterar el código existente. mediante **setReglas()**, evitando modificar la estructura interna de la clase base.
- **Composición y Agregación:** Estructura las relaciones esenciales del diseño. **Juego** mantiene una relación de composición con **Baraja** y **Mesa**, lo que significa que el ciclo de vida de estos componentes depende estrictamente de la existencia de la partida para que la simulación tenga sentido.

6. DIAGRAMA UML ACTUALIZADO



7. CONCLUSIÓN DEL EQUIPO

Al realizar esta segunda parte, entendimos el verdadero valor de una primera parte bien estructurada. Al contar con clases sólidas como **Baraja** y **Cartas**, no tuvimos que reinventar la lógica del código. En su lugar, pudimos concentrar el esfuerzo en las mecánicas lógicas, validación de estados y la experiencia del usuario interactuando con la consola. Mediante la herencia y el uso correcto de las colecciones dinámicas garantizamos un código limpio, mantenible y escalable para futuros desarrollos.